

Boolean Rule Language Compiler Test Cases Screenshots

Valid Test Cases:

Simple Comparison

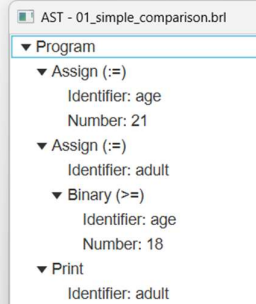
```

Enter test file path, or press Enter to type code manually, or enter 0 to exit:
tests\\valid\\01_simple_comparison.brl
Running file: tests\\valid\\01_simple_comparison.brl
Tokens:
ID('age')
ASSIGN(':=')
NUMBER('21')
SEMICOLON(';')
ID('adult')
ASSIGN(':=')
ID('age')
GE('>=')
NUMBER('18')
SEMICOLON(';')
PRINT('print')
ID('adult')
SEMICOLON(';')
EOF('')

Parse successful.
Program
  Assign (:=)
    Identifier: age
    Number: 21
  Assign (:=)
    Identifier: adult
    Binary (>=)
      Identifier: age
      Number: 18
  Print
    Identifier: adult

Traversal:
Assigned age = 21.0
Assigned adult = true
Print: true
Opening ast GUI

```



Boolean Assignment

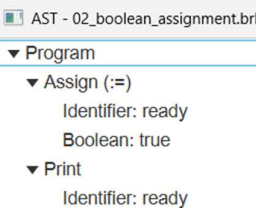
```

Enter test file path, or press Enter to type code manually, or enter 0 to exit:
tests\\valid\\02_boolean_assignment.brl
Running file: tests\\valid\\02_boolean_assignment.brl
Tokens:
ID('ready')
ASSIGN(':=')
TRUE('true')
SEMICOLON(';')
PRINT('print')
ID('ready')
SEMICOLON(';')
EOF('')

Parse successful.
Program
  Assign (:=)
    Identifier: ready
    Boolean: true
  Print
    Identifier: ready

Traversal:
Assigned ready = true
Print: true
Opening ast GUI

```



Team 7

AND NOT Precedence

Enter test file path, or press Enter to type code manually, or enter 0 to exit:

tests\\valid\\03_and_not_precedence.brl

Running file: tests\\valid\\03_and_not_precedence.brl

Tokens:

```
ID('approved')
ASSIGN(':=')
ID('income')
GT('>')
NUMBER('5000')
AND('&')
NOT('not')
ID('blocked')
SEMICOLON(';')
PRINT('print')
ID('approved')
SEMICOLON(';')
EOF('')
```

Parse successful.

Program

```
Assign (:=)
  Identifier: approved
Binary (and)
  Binary (>)
    Identifier: income
    Number: 5000
  Unary (not)
    Identifier: blocked
Print
  Identifier: approved
```

AST - 03_and_not_precedence.brl

▼ Program

▼ Assign (:=)

Identifier: approved

▼ Binary (and)

▼ Binary (>)

Identifier: income

Number: 5000

▼ Unary (not)

Identifier: blocked

▼ Print

Identifier: approved

Arithmetic Inside Comparison

Enter test file path, or press Enter to type code manually, or enter 0 to exit:

tests\\valid\\04_arithmetic_inside_comparison.brl

Running file: tests\\valid\\04_arithmetic_inside_comparison.brl

Tokens:

```
ID('valid')
ASSIGN(':=')
LPAREN('(')
ID('score')
PLUS('+')
ID('bonus')
RPAREN(')')
GE('>=')
NUMBER('60')
AND('&')
ID('attempts')
LT('<')
NUMBER('3')
SEMICOLON(';')
PRINT('print')
ID('valid')
SEMICOLON(';')
EOF('')
```

Parse successful.

Program

```
Assign (:=)
  Identifier: valid
Binary (and)
  Binary (>=)
    Binary (+)
      Identifier: score
      Identifier: bonus
    Number: 60
  Binary (<)
    Identifier: attempts
    Number: 3
Print
  Identifier: valid
```

AST - 04_arithmetic_inside_comparison.brl

▼ Program

▼ Assign (:=)

Identifier: valid

▼ Binary (and)

▼ Binary (>=)

▼ Binary (+)

Identifier: score

Identifier: bonus

Number: 60

▼ Binary (<)

Identifier: attempts

Number: 3

▼ Print

Identifier: valid

Team 7

OR AND Precedence

Enter test file path, or press Enter to type code manually, or enter 0 to exit:

tests\valid\05_or_and_precedence.brl

Running file: tests\valid\05_or_and_precedence.brl

Tokens:

```
ID('flag')
ASSIGN(':=')
TRUE('true')
OR('or')
FALSE('false')
AND('and')
NOT('not')
FALSE('false')
SEMICOLON(';')
PRINT('print')
ID('flag')
SEMICOLON(';')
EOF('')
```

Parse successful.

Program

```
Assign (:=)
  Identifier: flag
Binary (or)
  Boolean: true
  Binary (and)
    Boolean: false
  Unary (not)
    Boolean: false
Print
  Identifier: flag
```

Traversal:

Assigned flag = true

Print: true

Opening ast GUI

AST - 05_or_and_precedence.brl

▼ Program

▼ Assign (:=)

Identifier: flag

▼ Binary (or)

Boolean: true

▼ Binary (and)

Boolean: false

▼ Unary (not)

Boolean: false

▼ Print

Identifier: flag

Multiple Assignments

```
Tokens:
ID('base')
ASSIGN(':=')
NUMBER('10')
SEMICOLON(';')
ID('limit')
ASSIGN(':=')
ID('base')
MUL('*')
NUMBER('2')
PLUS('+')
NUMBER('5')
SEMICOLON(';')
ID('allowed')
ASSIGN(':=')
ID('limit')
GE('>=')
NUMBER('20')
AND('and')
NOT('not')
FALSE('false')
SEMICOLON(';')
PRINT('print')
ID('allowed')
SEMICOLON(';')
EOF('')
```

Parse successful.

Program

```
Assign (:=)
  Identifier: base
  Number: 10
Assign (:=)
  Identifier: limit
  Binary (+)
    Binary (*)
      Identifier: base
      Number: 2
    Number: 5
Assign (:=)
  Identifier: allowed
  Binary (and)
    Binary (>=)
      Identifier: limit
      Number: 20
    Unary (not)
      Boolean: false
Print
  Identifier: allowed
```

Traversal:

Assigned base = 10.0

Assigned limit = 25.0

Assigned allowed = true

AST - 06_multiple_assignments.brl

▼ Program

▼ Assign (:=)

Identifier: base

Number: 10

▼ Assign (:=)

Identifier: limit

▼ Binary (+)

▼ Binary (*)

Identifier: base

Number: 2

Number: 5

▼ Assign (:=)

Identifier: allowed

▼ Binary (and)

▼ Binary (>=)

Identifier: limit

Number: 20

▼ Unary (not)

Boolean: false

▼ Print

Identifier: allowed

Team 7

Nested Parentheses

Enter test file path, or press Enter to type code manually, or enter 0 to exit:

tests\valid\07_nested_parentheses.brl

Running file: tests\valid\07_nested_parentheses.brl

Tokens:

```
ID('status')
ASSIGN(':=')
NOT('not')
LPAREN('(')
LPAREN('(')
ID('points')
PLUS('+')
ID('bonus')
RPAREN(')')
LT('<')
ID('threshold')
RPAREN(')')
OR('or')
ID('override')
SEMICOLON(';')
PRINT('print')
ID('status')
SEMICOLON(';')
EOF('')
```

Parse successful.

Program

```
Assign (:=)
  Identifier: status
  Binary (or)
    Unary (not)
      Binary (<)
        Binary (+)
          Identifier: points
          Identifier: bonus
          Identifier: threshold
        Identifier: override
      Print
        Identifier: status
```

AST - 07_nested_parentheses.brl

▼ Program

▼ Assign (:=)

Identifier: status

▼ Binary (or)

▼ Unary (not)

▼ Binary (<)

▼ Binary (+)

Identifier: points

Identifier: bonus

Identifier: threshold

Identifier: override

▼ Print

Identifier: status

Complex Expression

Running file: tests\valid\08_complex_expression.brl

Tokens:

```
ID('eligible')
ASSIGN(':=')
LPAREN('(')
LPAREN('(')
ID('income')
PLUS('+')
ID('savings')
RPAREN(')')
GT('>')
ID('debt')
RPAREN(')')
AND('and')
LPAREN('(')
LPAREN('(')
ID('age')
PLUS('+')
NUMBER('1')
RPAREN(')')
GE('>=')
NUMBER('18')
OR('or')
ID('guardian')
RPAREN(')')
SEMICOLON(';')
PRINT('print')
ID('eligible')
SEMICOLON(';')
EOF('')
```

Parse successful.

Program

```
Assign (:=)
  Identifier: eligible
  Binary (and)
    Binary (>)
      Binary (+)
        Identifier: income
        Identifier: savings
      Identifier: debt
    Binary (or)
      Binary (>=)
        Binary (+)
          Identifier: age
          Number: 1
        Number: 18
        Identifier: guardian
      Print
        Identifier: eligible
```

AST - 08_complex_expression.brl

▼ Program

▼ Assign (:=)

Identifier: eligible

▼ Binary (and)

▼ Binary (>)

▼ Binary (+)

Identifier: income

Identifier: savings

Identifier: debt

▼ Binary (or)

▼ Binary (>=)

▼ Binary (+)

Identifier: age

Number: 1

Number: 18

Identifier: guardian

▼ Print

Identifier: eligible

Sample

```

Tokens:
ID('ziad')
ASSIGN(':=')
NUMBER('100000')
SEMICOLON(';')
ID('base')
ASSIGN(':=')
ID('ziad')
DIV('/')
NUMBER('5')
SEMICOLON(';')
ID('allowed')
ASSIGN(':=')
LPAREN('(')
ID('base')
MUL('*')
NUMBER('2')
PLUS('+')
NUMBER('5')
RPAREN(')')
GE('>=')
NUMBER('20')
AND('&')
NOT('not')
FALSE('false')
SEMICOLON(';')
PRINT('print')
ID('allowed')
SEMICOLON(';')
EOF('')

Parse successful.
Program
  Assign (:=)
    Identifier: ziad
    Number: 100000
  Assign (:=)
    Identifier: base
  Binary (/)
    Identifier: ziad
    Number: 5
  Assign (:=)
    Identifier: allowed
  Binary (and)
    Binary (>=)
      Binary (+)
        Binary (*)
          Identifier: base
          Number: 2
        Number: 5
      Number: 20
    Unary (not)
      Boolean: false
  Print
    Identifier: allowed

Traversal:
Assigned ziad = 100000.0
Assigned base = 20000.0
Assigned allowed = true
Print: true

```

AST - sample.brl

▼ Program

▼ Assign (:=)

Identifier: ziad

Number: 100000

▼ Assign (:=)

Identifier: base

▼ Binary (/)

Identifier: ziad

Number: 5

▼ Assign (:=)

Identifier: allowed

▼ Binary (and)

▼ Binary (>=)

▼ Binary (+)

▼ Binary (*)

Identifier: base

Number: 2

Number: 5

Number: 20

▼ Unary (not)

Boolean: false

▼ Print

Identifier: allowed

Team 7

Invalid Test Cases:

Missing Semicolon

Enter test file path, or press Enter to type code manually, or enter 0 to exit:

tests\invalid\01_missing_semicolon.brl

Running file: tests\invalid\01_missing_semicolon.brl

Tokens:

```
ID('adult')
ASSIGN(':=')
ID('age')
GE('>=')
NUMBER('18')
PRINT('print')
ID('adult')
SEMICOLON(';')
EOF('')
```

Syntax error at position 20: Expected ';' after statement. Found 'print'.

Missing Identifier After Print

Enter test file path, or press Enter to type code manually, or enter 0 to exit:

tests\invalid\02_missing_identifier_after_print.brl

Running file: tests\invalid\02_missing_identifier_after_print.brl

Tokens:

```
PRINT('print')
SEMICOLON(';')
EOF('')
```

Syntax error at position 6: Expected identifier after 'print'. Found ';'.

Missing Assignment Expression

Enter test file path, or press Enter to type code manually, or enter 0 to exit:

tests\invalid\03_missing_assignment_expression.brl

Running file: tests\invalid\03_missing_assignment_expression.brl

Tokens:

```
ID('ready')
ASSIGN(':=')
SEMICOLON(';')
EOF('')
```

Syntax error at position 9: Expected expression. Found ';'.

Team 7

Unclosed Parenthesis

Enter test file path, or press Enter to type code manually, or enter 0 to exit:
tests\invalid\04_unclosed_parenthesis.brl

Running file: tests\invalid\04_unclosed_parenthesis.brl

Tokens:

```
ID('valid')
ASSIGN(':=')
LPAREN('(')
ID('score')
PLUS('+')
ID('bonus')
GE('>=')
NUMBER('60')
SEMICOLON(';')
PRINT('print')
ID('valid')
SEMICOLON(';')
EOF('')
```

Syntax error at position 29: Expected ')' after expression. Found ';'.

Dangling AND Operator

Enter test file path, or press Enter to type code manually, or enter 0 to exit:
tests\invalid\05_dangling_and_operator.brl

Running file: tests\invalid\05_dangling_and_operator.brl

Tokens:

```
ID('approved')
ASSIGN(':=')
ID('income')
GT('>')
NUMBER('5000')
AND('and')
SEMICOLON(';')
PRINT('print')
ID('approved')
SEMICOLON(';')
EOF('')
```

Syntax error at position 30: Expected expression after operator 'and'. Found ';'.

Team 7

Expression Starts With Operator

Enter test file path, or press Enter to type code manually, or enter 0 to exit:

tests\invalid\06_expression_starts_with_operator.brl

Running file: tests\invalid\06_expression_starts_with_operator.brl

Tokens:

ID('flag')

ASSIGN(':=')

OR('or')

TRUE('true')|

SEMICOLON(';')

PRINT('print')

ID('flag')

SEMICOLON(';')

EOF('')

Syntax error at position 8: Expression cannot start with operator 'or'. Found 'or'.

Missing RHS Of Comparison

Enter test file path, or press Enter to type code manually, or enter 0 to exit:

tests\invalid\07_missing_rhs_of_comparison.brl

Running file: tests\invalid\07_missing_rhs_of_comparison.brl

Tokens:

ID('adult')

ASSIGN(':=')

ID('age')

GE('>=')

SEMICOLON(';')

PRINT('print')

ID('adult')

SEMICOLON(';')

EOF('')

Syntax error at position 16: Expected expression after operator '>='. Found ';'.

Team 7

Double Semicolon

Enter test file path, or press Enter to type code manually, or enter 0 to exit:

tests\invalid\08_double_semicolon.br1

Running file: tests\invalid\08_double_semicolon.br1

Tokens:

ID('ready')

ASSIGN(':=')

TRUE('true')

SEMICOLON(';')

SEMICOLON(';')

PRINT('print')

ID('ready')

SEMICOLON(';')

EOF('')

Syntax error at position 14: Empty statements are not allowed. Found ';'.

Adjacent Operands

Enter test file path, or press Enter to type code manually, or enter 0

tests\invalid\09_adjacent_operands.br1

Running file: tests\invalid\09_adjacent_operands.br1

Tokens:

ID('value')

ASSIGN(':=')

ID('age')|

NUMBER('18')

SEMICOLON(';')

PRINT('print')

ID('value')

SEMICOLON(';')

EOF('')

Syntax error at position 13: Expected ';' after statement. Found '18'.

Invalid Character

Enter test file path, or press Enter to type code manually, or enter 0 to exit:

tests\invalid\10_invalid_character.br1

Running file: tests\invalid\10_invalid_character.br1

Lexical error at position 17

Team 7

Malformed Operator

Enter test file path, or press Enter to type code manually, or enter 0 to exit:

tests\invalid\l1_malformed_operator.brl

Running file: tests\invalid\l1_malformed_operator.brl

Tokens:

ID('check')

ASSIGN(':=')

ID('age')

LT('<')

GT('>')

NUMBER('18')

SEMICOLON(';')

PRINT('print')

ID('check')

SEMICOLON(';')

EOF('')

Syntax error at position 14: Expected expression after operator '<'. Found '>'.

Keyword As Identifier

Enter test file path, or press Enter to type code manually, or enter 0 to exit:

tests\invalid\l2_keyword_as_identifier.brl

Running file: tests\invalid\l2_keyword_as_identifier.brl

Tokens:

AND('and')

ASSIGN(':=')

TRUE('true')

SEMICOLON(';')

PRINT('print')

AND('and')||

SEMICOLON(';')

EOF('')

Syntax error at position 0: Keywords cannot be used as statement identifiers. Found 'and'.